Project for CMPS 497/CMPE 471 Deep Learning

Cross-prompt Automated Essay Scoring using Neural Models (Ver. 3)

Changes in Ver. 3

- The updated sections have their **titles highlighted**.
- A couple of highlighted sentences have also been updated in the "Very important notes" under "Introduction" sec 1.

1. Introduction

The objective of this project is to apply what you have learned in this course to tackle an example of a *real-world* problem, which is **Automated Essay Scoring** (AES), by designing, training, and evaluating several deep learning models for it.

We will follow some of the work described in the attached paper titled "Conundrums in Cross-Prompt Automated Essay Scoring: Making Sense of the State of the Art" that was recently published at ACL 2024, the top conference in the Natural Language Processing (NLP) field. Yes! you will be able to replicate (some of) such work with just what you have studied in this course 🙂.

Very important notes

Before you start, here are some very important notes for you to consider.

- This version of the project description is **ver. 3, which is the final version**.
- Please start **VERY EARLY!**
- Read this document very carefully and ask if you have any questions in class or during the office hours.
- You will use **only** the **PyTorch** framework to build, train, and evaluate your models for all approaches, in addition to **Hugging Face** for approach C only.
- This is a team assignment. You can (of course) distribute the work within the team, BUT your submitted work is the responsibility of the whole team. You are encouraged to work together when it comes to the implementation design, the setup of the experiments, and the analysis of the results.
- You are given some starter code to (1) read the data including the essays and the features, and (2) to evaluate the model outputs using a suitable measure for AES called Quadratic Weighted Kappa (QWK).
- Consider the "Important Issues to Think About" (listed in Section 7) very seriously.
- Note that a question (or more) on the project will be included in the final exam.

Academic dishonesty

Plagiarism of any kind (from another group, from the internet, from ChatGPT or alike, or from any external source) will result in a grade of zero, and your case will be reported to the university. Cheating or attempting to cheat are academic violations according to Qatar University rules and regulations, and in some cases, they may result in final dismissal from the University. Students should not under any circumstances commit or participate in any cheating attempt or any act that violates student code of conduct.

2. A Real-World Problem: Automated Essay Scoring

Writing an essay is a task that is usually included in several language exams (e.g., IELTS or GRE) or in language classes in high schools, where a student is required to write an essay on a *topic* that is described by a given prompt to measure the writing capabilities of the student.

Prompt

A prompt is a paragraph or so that describes the topic of the essay and provides some instructions about writing it. An example prompt is given below.

> "*Write about patience. Being patient means that you are understanding and tolerant. A patient person experiences difficulties without complaining. Do only one of the following: write a story about a time when you were patient OR write a story about a time when someone you know was patient OR write a story in your own way about patience*."

Automated Essay Scoring (AES)

Automated essay scoring (AES) is the task of automatically scoring the quality of the written essay. A single score (known as the ***holistic score***) can be assigned to summarize the overall quality of the essay, as shown in Figure 1. This is called ***holistic scoring***.

Figure 1. Holistic Scoring

However, an overall score might not be very useful in indicating how a student can improve. Alternatively, there are several essay traits (i.e., dimensions of essay quality, such as organization, word choice, or sentence fluency) that can be measured to better provide feedback to the students on specific weaknesses and strengths. Scoring such individual dimensions is called ***trait scoring***, as shown in Figure 2.

Figure 2. Trait Scoring

Prompt-specific AES

Traditional work on AES has focused on ***prompt-specific scoring***, where an AES model is trained on manually-annotated essays written for a given prompt and subsequently applied to essays written for the same prompt, as shown in Figure 3.

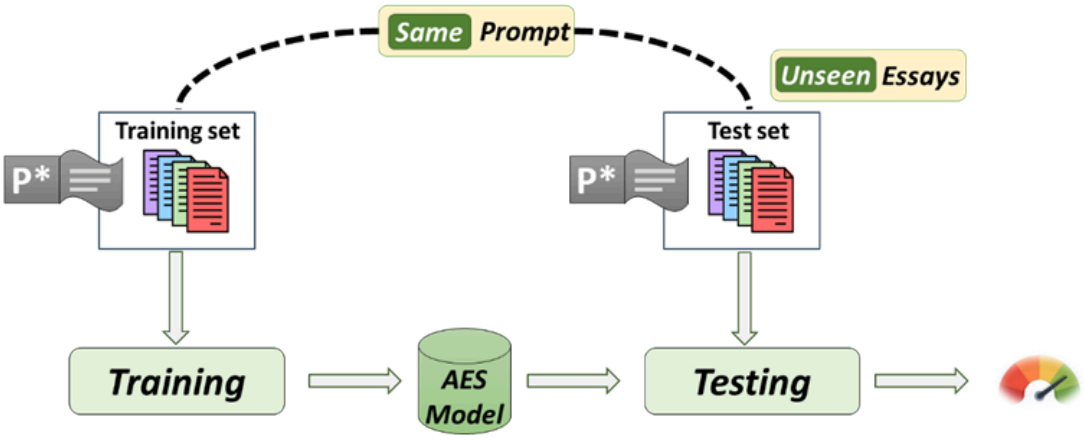

Figure 3. Learning a Prompt-specific AES model for prompt P*.

Although a reasonable success has been made on prompt-specific scoring, there is a key weakness associated with these models. A prompt-specific model is meant to score essays that are written for *one specific prompt*; when it is applied to essays written for a *different* prompt, its performance often deteriorates considerably. Hence, in practice, before such models are applied to score essays written for a new prompt, they need to be retrained on a relatively large number of manually-scored essays written for that new prompt. However, manually scoring essays is a time-consuming process and requires a lot of expertise and time.

Cross-prompt AES

To address the aforementioned weakness, researchers have begun working on ***cross-prompt essay scoring***, where the goal is to train a model on annotated essays that are *not* written for the target prompt and then apply the resulting model to essays written for a new (target) prompt that was never seen, as shown in Figure 4. In other words, the goal of *cross-prompt scoring* is

to train a model on essays written for existing prompts so that it can accurately score essays written for a new prompt without the need to retrain it on essays from the new prompt. This is much more practical and closer to the real scenario.

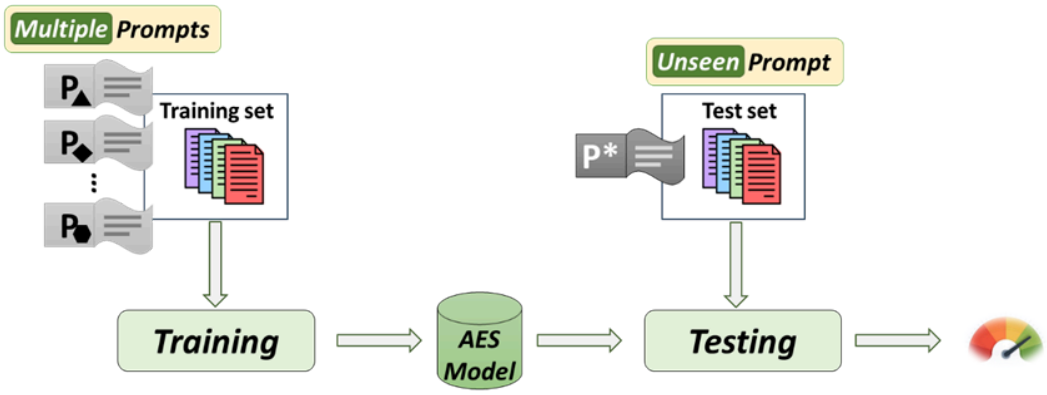

Figure 4. Learning a Cross-prompt AES model.

However, cross-prompt scoring is a very challenging task. To understand why, consider the task of scoring essays written for the prompt "*Write a persuasive essay on why one should (or should not) support multidisciplinary education*". Intuitively, a high-scoring essay should provide evidence(s) that can adequately support the claim of why one should (or should not) support multidisciplinary education. However, determining whether an argument is persuasive could require domain knowledge (in this case knowledge about multidisciplinary education), which the model may not possess in the absence of training data for the new prompt.

3. Overview of the Project

In this project, you will focus on building **cross-prompt models** for AES. You will view the AES problem as a regression problem, where the output of the model is numeric. We aim to answer the following question.

> *Can we achieve state-of-the-art Cross-prompt AES performance if we use (1) simple neural architectures like feed-forward neural networks adopting a purely feature-based approach, and (2) encoder-based transformers architecture like BERT adopting a mixed feature-and-representation-based approach?*

To answer that question, you will train and test several models using **two different ways**:

- training a prediction model for holistic scoring
- training a joint model that predicts the holistic and trait scores together.

You will experiment with **two different architectures**:

- Fully-connected neural networks (FNN)
- Encoder-based transformers.

You will evaluate your cross-prompt AES models on a standard evaluation dataset for AES research. Details are presented in the following sections.

4. Data and Features

For model training and evaluation, we employ the widely-used ASAP dataset and its extension, ASAP++.

ASAP

ASAP (Automated Student Assessment Prize), which was released as part of a Kaggle competition in 2012, is composed of essays manually annotated with their holistic scores. The essays are written for eight prompts, including two for persuasive essays, two for narrative essays, and four for source-dependent essays. Different rubrics are used for scoring prompts, and as a result, the score ranges for different prompts can be different. For instance, one prompt has a score range of 0 to 3, while another prompt has a score range of 1 to 6. The eight prompts and their statistics can be found in Table 4 (in Appendix A) of the attached paper.

ASAP++

ASAP++ is an extension of ASAP where each essay is additionally scored along different traits. Eight traits are scored, including:

- *Content*: how clear and focused the writing is and how well-developed the main ideas are.
- *Word Choice*: how well the words convey the intended message.
- *Organization*: how well-organized the essay is.
- *Prompt Adherence*: how adherent the essay is to the prompt.
- *Sentence Fluency*: whether the sentences in the essay are of high quality.
- *Conventions*: how well the essay demonstrates standard writing conventions.
- *Narrativity*: how coherent and cohesive the response is.
- *Language*: how good grammar and spelling are.

Note, however, that the traits used for essays written for different prompts can be different: some essays are scored along three traits, while others are scored along five traits. Additional information on the traits can be found in Tables 5 and 6 (in Appendix B) of the attached paper.

Input Features

We will employ existing features taken from Ridley et al.'s (2020) system. These features are a manually curated set of 86 prompt-independent features that include readability features, text complexity features, text variation features, length-based features, and sentiment-based features. More details on these features are given in the attached paper in Table 7 in Appendix C. So, you will assume that the number of inputs is 86 (unless otherwise stated).

Note that you don't need to manipulate/process any of the input features. They are already processed (and normalized) for you; just use them as is.

Data and Starter code

You are given two files:

- **`dataset.csv`**: a CSV file that has all essays, their holistic and trait scores, and their features. Each row represents one essay. The columns are (in order):
  - `essay_id`: the unique identifier of the essay
  - `prompt_id`: the unique identifier of the prompt the essay is written for
  - `essay_text`: the text content of the essay
  - `holistic:` the holistic score of the essay
  - `content`: the content score of the essay
  - `organization`: the organization score of the essay
  - `word_choice`: the word choice score of the essay
  - `sentence_fluency`: the sentence fluency score of the essay
  - `conventions`: the conventions score of the essay
  - `prompt_adherence`: the prompt adherence score of the essay
  - `language`: the language score of the essay
  - `narrativity`: the narrativity score of the essay
  - finally, **86 columns** representing the 86 feature values extracted from the essay
- **`general_utils.py`**: a python script that has some useful utilities you should use.
  - `SCORE_RANGES` is a dictionary that has the ranges of the holistic and trait scores for each prompt.
  - `read_data(path)`: a function that reads the CSV dataset file and returns a dictionary that has ***parallel*** lists of values for the different columns.
  - `quadratic_weighted_kappa(y_true, y_pred)`: a function that calculates the Quadratic Weighted Kappa (QWK) score between the true scores and the model predictions.

5. Models

You will experiment with the following approaches.

Approach A: Holistic scoring using FFN

In this approach, we will build a model for holistic scoring. It takes as input the set of 86 features and outputs a single score. The model is based on shallow/deep fully-connected neural networks and is trained and tested using the provided dataset.

Approach B: Joint model for holistic and trait scoring using FFN

In this approach, we will build a model that jointly predicts all of the nine scores: the holistic score and the eight trait scores, in a multi-task learning paradigm. The intuition is that the holistic score naturally depends on the traits, therefore, learning the trait scores together with the holistic score might benefit each other, resulting in a better prediction model.

Similar to Approach A, the model is based on shallow/deep fully-connected neural networks, but here it takes as input the set of 86 features and outputs nine scores, i.e., the output layer of the network contains nine (rather than one) output nodes. Specifically, there is one output node for predicting each of the eight traits and one output node for predicting the holistic score. This is considered a simple example of multi-task learning.

Approach C (BONUS): Holistic scoring using BERT

In this approach, we will build a model for holistic scoring (similar to Approach A) using BERT (an encoder-only transformer-based model). We will upload the pre-trained model, then fine-tune it for AES. To fine-tune it, you will add a "***regression head***" (on top of BERT encoder stack) that takes the CLS embeddings (the output embedding vector corresponding to the CLS token) of the essay *concatenated with* the set of 86 features of the essay, and outputs a single score. The regression head for this project can either be one output node, with no hidden layers, or one hidden layer of 4 or 8 hidden units before the output node. **For simplicity, you will build models only for one target prompt, prompt 1**.

For this approach, in addition to PyTorch, we will use the **Hugging Face** platform (a platform that facilitates loading and using transformer-based models through their `Transformers` library). A helpful guide on using BERT for classification or regression tasks can be found here. **You need to follow the relevant steps there**.

6. Evaluation

Experimental setup

You will conduct cross-prompt evaluation via ***leave-one-prompt-out*** cross-validation experiments. Since ASAP/ASAP++ contains eight prompts, we divide the essays into eight folds based on the prompt for which an essay is written. For each target prompt (fold) experiment, you will use that fold (the essays written for that prompt) for testing (keep it unseen), and the remaining seven folds (the essays written for other prompts) for model training and hyper-parameter tuning. Then, for each target prompt, you will actually do 7-fold cross-validation to tune your hyper-parameters.

Evaluation metric

We employ Quadratic Weighted Kappa (QWK) as our metric for holistic and trait-specific scoring. Since QWK is an agreement metric, higher values are better. You will be given code that implements QWK.

Training/Hyper-parameter tuning (Approaches A & B)

You will train your models while tuning/optimizing the hyper-parameters. Notice that you will need to tune the hyper-parameters separately for each target prompt using 7-fold cross-validation (i.e., over all prompts except the target prompt). The training and tuning process will go over 2 steps.

1. Tuning some hyper-parameters using Grid Search while fixing the others.
2. Optimizing the batch size.

To initialize all your models, use He initialization.

Step 1: Grid Search

While fixing the values of some hyper-parameters (see below), you will use Grid Search (i.e., trying all combinations) to tune (i.e., find the best values of) the following hyper-parameters:

- **Number of hidden layers**: The possible values are 1, 2, 4, and 8.
- **Number of hidden units per layer**: We will fix the number of hidden units per layer. The possible values are 8, 16, and 32.
- **Learning rate**: The possible values are 0.001, 0.01, and 0.1.
- **Early Stopping**: We will fix the maximum number of epochs to 15, but you need to do early stopping.

There are other hyper-parameters that we will fix in the project:

- We will fix the batch size to 4 **initially**. In step 2, you will optimize it.
- We will only use ReLU activation function for all hidden layers.
- We will use one loss function for Approach A and another for Approach B. You will choose/design them and justify them in the report.
- We will **not** use dropout.
- We will use AdamW optimizer with B_1 = 0.9 and B_2 = 0.999.
- L2 regularization: We will use L2 regularization with λ=0.1.

Step 2: Optimizing the Batch Size

After you tune the above hyper-parameters (i.e., finding the best value for each of the ones you changed), you will tune the **batch size** hyper-parameter. The possible values are 4, 8, 16 and 32, while fixing the values of all other hype-parameters.

Fine-tuning/Regression head tuning (Approach C)

You will fine-tune your BERT-based model while tuning the design of the regression head. All other hyper-parameters will be fixed for this approach (see below). Similar to approaches A & B, you will need to tune the design of the regression head for the target prompt 1 using 7-fold cross-validation (i.e., over all prompts except the target prompt 1),.

You will try three options for the regression head:

I. just one output node for predicting the holistic score, with no hidden layers,

II. one hidden layer of 4 hidden units before the output node, and
III. one hidden layer of 8 hidden units before the output node.

The other hyper-parameters will be fixed as follows:

- Set batch size to 32.
- Use ReLU activation function for the hidden layer.
- Use the same loss function used for Approach A.
- We will **not** use dropout.
- Use AdamW optimizer with B_1 = 0.9 and B_2 = 0.999.
- Use L2 regularization with λ=0.1.
- Use a learning rate of 0.005.
- Fix the number of epochs to 5 (no early stopping).

Important Notes

- Remember that for this approach, you will build models *only for one target prompt*, which is Prompt 1.
- For fine tuning, the BERT model will be initialized with the pre-trained model and fine-tuned entirely. We will use the "`bert-base-uncased`" model (110M parameters).
- For fine-tuning a (relatively) large model like BERT, you will definitely need a machine with a GPU. So, you are strongly encouraged to use Google Colab for that approach.
- The regression head will be initialized using **HE initialization**.

Saving and loading models

Approaches A & B

For saving a model `model` to a file at `file_path`, you need to use the following code.

```
scripted_model = torch.jit.script(model)
scripted_model.save(file_path)
```

For loading a model `model` from a file at `file_path`, you need to use the following code.

```
model = torch.jit.load(file_path)
```

Please stick to the above code for saving and loading models.

Approach C

For approach C, you will need to find a suitable way to share your final models. Note that the size of those models is now much larger than the ones from approaches A and B due to the pre-trained model.

7. Important Issues to Think About

- You need to design the appropriate loss function for each approach carefully. In your report, you will have to justify the design decisions and choices. The choice of the loss functions is **not** a hyper-parameter in this project.

- Notice that since not all traits are applicable to all prompts, any score predicted for an inapplicable trait should not contribute to the loss.
- While we aim to minimize the loss on the training data, the AES performance is measured by QWK (check the evaluation section above). Therefore, you need to think about how you will measure the performance of your models during validation and test phases.
- Score ranges for different prompts might be different. Therefore, you might need to scale all the scores to one range for training and scale them back for evaluation.
- To start, work only on the holistic scoring for one target prompt. Once you do that, repeat for other target prompts. That means that your code should be reusable and generic enough to do that.

8. Submission and Grading

Your submission will be graded based on (1) the code you wrote, (2) the models you trained and their performance, (3) the report you will submit, and (4) an oral discussion with your entire team. The grade is out of **60 points**.

Stages and timeline

There are **two stages of submissions**:

- **Stage 1** [12 points]: Initial Submission (**due Sat Nov 16th**):
  You will submit **only** your work on **approach A**.
- **Stage 2** [48 points for approaches A & B + 12 points BONUS for approach C]: Final Submission (**due Sat Dec 7th**):
  You will submit your work on **all approaches** including any updates on approach A.

What to Submit

For **each** stage, you will submit three things to blackboard (submission by any other means is **not** accepted):

1. **Report:** Check the "Report" subsection below for details.
2. **Code**:
   Wrap up all your scripts **for each approach separately in one zip file** called `GX_models_P.zip` (where `X` denotes your group number, and `P` denotes the name of the approach). Your code should be clean and well-documented. Along with the code, you will submit a guide (like a README) on how to run your code and replicate all of your experiments.
3. **Models**: **For approach A or B**, you will submit 8 different models, one trained for each target prompt, except **approach C**, where you will submit one model trained for prompt 1. Additionally, for each approach, you will submit one extra "deployed" model that will be used to grade essays on unseen prompts at our end.

- Wrap up all your *final* models **for each approach separately in one zip file** called `GX_models_P.zip` (where `X` denotes your group number, and `P` denotes the name of the approach).
    - Name the files of your individual models as `model-n-k.pt`, where `n` is the approach (A, B, etc.), and `k` is the target prompt (1, 2, etc.). For example, `model-B-3.pt` is the model of approach B for the target prompt 3.
    - Name the deployed model as `model-n-deploy.pt`, where `n` is the approach (A, B, etc.).

Note that Only the ***lead*** of the group has to submit, **not all** group members.

Submitting models for "deployment"

You will submit one final additional model for each approach that can act as a model for "deployment". We will evaluate those models to grade essays on unseen prompts that are not part of the data you get. The **best 3 models** for each approach will get **bonus points**.

Oral Discussion

There will be an in-person discussion with the entire team *after the final submission*. It is the responsibility of every single member of the team to understand every single line of the code and the report. The group might be asked questions about the project or their submitted work and/or asked to run/modify any of the submitted code. Dates, times, and locations will be announced later.

Each member of the team will be assigned a *discussion score*, which is a score or weight between 0 and 1 based on his/her responses and understanding in the discussion. The final grade will be computed as follows:

*Final grade = discussion_score * (stage1-grade + stage2-grade)*

Report

Stage 1: Initial Submission for Approach A

For stage 1, you will submit a report that covers **(1) Models and Training, (2) Experimental Evaluation, and (3) Student Contributions (so far)**:

Models and Training

You need to (at least) discuss the following:

- How did you split the data for cross-validation?
- How did you train the models you used for the test sets?
- How did you train the model(s) for deployment?
- How did you design the loss function? Justify your decisions.
- How did you handle the different score ranges?

Experimental Evaluation

You need to report and discuss the following:

- Performance per target prompt (validation vs. test).
    - Main observation on the results
    - Is there any discrepancy of model performance across prompts. Why?
- Best hyper-parameter values per target prompt.
    - Discussion on the best hyper-parameters
- Effect of changing the batch size.
- Performance comparison with state-of-the-art models (check the attached paper).

Student Contributions (so far)

You need to report the responsibilities and contributions of each member of the group so far.

Stage 2: Final Submission for All Approaches

For the final submission, you will submit a report that covers **(1) Models and Training, (2) Experimental Evaluation, and (3) Student Contributions for all approaches.**

Models and Training

Overall, you need to (at least) discuss the following:

- How did you split the data for cross-validation?
- How did you train the models you used for the test sets?
- How did you train the model(s) for deployment?
- How did you handle the different score ranges?

For approaches A & B, you need to discuss the following:

- How did you design the loss function? Justify your decisions.
- How did you initialize the parameters?

For approach C, you need to discuss the following:

- How did you use both the CLS embeddings and essay features?
- How did you initialize the parameters?

Experimental Evaluation

For approaches A & B, you need to report and discuss the following:

- Performance per target prompt (validation vs. test).
    - Main observation on the results
    - Is there any discrepancy of model performance across prompts. Why?
- Best hyper-parameter values per target prompt.
    - Discussion on the best hyper-parameters
- Effect of changing the batch size.
- Performance comparison with state-of-the-art models (check the attached paper).

For approach C, you need to report and discuss the following:

- Performance on Prompt 1 (validation vs. test).
    - Main observation on the results
- Best configuration of the regression head.
    - Discussion on the best hyper-parameters
- Performance comparison with state-of-the-art models (check the attached paper).

Student Contributions

You need to report all the responsibilities and contributions of each member of the group **for each approach separately**.